Spiff v2 — Design notes

What changed between the v1 mockup and v2, screen by screen, and the decisions taken along the
way. Written 31 August 2026. Companion to Spiff-v2-Research.md, which explains why.

The shape of the change

v1 answered "can a person ask a question and get a governed answer back?" It ended with nine screens, a global chat drawer, and three governance promises stated in modal copy.

v2 answers the question a sceptic asks next: "where did that number come from, who else can see it, and who decided what it means?" That is a different product surface — a catalogue, a definitions layer, a rules layer, an access layer and an audit layer — so v2 roughly doubles the screen count and adds a data spine underneath all of it.

Four structural changes:

- One file became a source tree. v1 was a single 1.5 MB HTML file. v2 is src/ — head, two stylesheets, shell, the v1 engine, a v2 UI kit, seven data modules and one file per view — concatenated by build.mjs. CONTRACT.md is the build contract every view module is written against.
- A shared data spine. src/data/50-org.js holds the org (428 users, 9 regions, 7 divisions), the current user, 6 systems of record, 8 role bundles, 12 groups, 32 named people plus 396 unlisted, 15 datasets with full field lists, and a 6-term glossary. Every v2 screen reads from it, so the same person, dataset and rule appear consistently everywhere.
- A second UI kit. core/45-ui2.js adds panels, KPIs, callouts, tabs, switches, three-state permission controls, meters, rings, certification and sensitivity badges, a page-head helper, a modal helper and the viewer simulator. v1's components still work; nothing was renamed.
- Governance moved from modal copy into chrome. The top bar carries a persistent Scoped to you chip, the user chip reads Scoped · 4 of 6 regions, and a full-width warning bar appears whenever you are previewing as someone else.

Screen by screen

Carried over from v1, unchanged in intent

Screen	Note
Ask	Same front door and the same plain-language framing. Suggestion chips now come from the viewer's own entitlements.
Team spaces	Unchanged; each space still scopes its own systems.
Library	Unchanged.
Dashboards	Unchanged, including refresh cadence set at publish time.
Triggers & schedules	Unchanged, plus a plain-language gating condition ("only send when this is true").

Changed

Screen	What changed and why
Answer	The parsed question now sits above the chart as a row of coloured chips — measure, grouping, filter, timeframe — and each follow-up highlights the chip that changed. v1 put provenance in a footer; a wrong answer was only visibly wrong after you read the chart. Chart-type eligibility is unchanged, with one rule added: keep the chart on follow-ups and drills unless the data shape forbids it.

My workspace	Saved items now include saved views of a shared answer — filter state kept, still synced to the original — with an unsaved-change dot and an Update action rather than a save bar.
Automations	Reorganised on a configuration / schedule / history split, and the schedule dialog now lists what each recipient will actually receive. Group subscriptions gained a personal unsubscribe and a "subscribed via LDM Coordinators" line.
Capabilities	Capabilities you cannot use are shown with the reason, not hidden.
Chat drawer	Still there, still collapsible, still auto-groups by theme — but it is no longer the only place a conversation lives (see Portal chat).

New in v2

Screen	What it is
Home	A find surface, not a dashboard: search, what you run, what colleagues run, what changed. Every item re-runs per viewer.
Onboarding & registration	Four steps on sample data, skippable from screen one, ending with one report the user owns and follows. "Connect your own data" comes last.
Portal chat	The three-panel workspace: the plan (what Spiff will do), definitions and instructions (what it can see), reachable data (what it can reach, active connectors only). Approval mode sits in the composer.
Data catalogue	15 datasets, filterable by domain, sorted by how much they are actually used.
Sources	The registry of raw material Spiff has been pointed at — nine sources across five kinds — with what each one taught it, what it still cannot explain, and which catalogue datasets trace to no source at all.
Understanding run	One read of one source, laid out as what it opened, what it concluded and the proof. Every conclusion is a proposal a named person accepts, edits or rejects.
Dataset profile	The heart of v2. Identity, trust and freshness, stewardship, fields with examples and masking state, sample rows under your own policies, scope and coverage, rules in force, usage, access and changelog.
Definitions / glossary	One agreed definition per term, with owner, version and the date it was agreed.
Business rules	Rules written as sentences with dropdowns, a named hit policy for overlaps, tolerance, and a one-click preview.
People & access	Groups and role bundles, never individual grants; an approver queue; a review queue that shows the diff and flags dormancy.
My data access	What you can see, what you asked for, and where each request has got to.
Connectors (MCP)	The full tool list before you connect, read split from write, three permission states per tool, and an explicit statement of what Spiff will never do.
Activity log	Reads and exports alongside writes, denials alongside grants, run-as identity on every row.
Viewer simulator	Promoted from a v1 "next idea" to a real screen, reachable from the share dialog, the rule editor, a dataset and a person.

Sources — where the meaning comes from

Oren extended the brief mid-build:

> "the system needs to be able to register data sources - e.g. database access, github access,
> documents, PDFs, excel sheets etc. then we ask the system to understand datasets by reading the
> rules from the code, the logic from the code, understand the database structure from the code and
> documentation etc."

The shift it represents

Everything v2 had built until then rested on an assumption nobody had said out loud: that the datasets already exist, and that somebody has already written down what they mean. The catalogue
lists fifteen datasets with grain statements, field descriptions and rules in force. The Dataset profile is confident that one row is one member per meeting occurrence. The glossary carries an agreed definition of Attendance — version 3, agreed 12 April 2026, owned by the Head of Statistics.

None of that arrives by itself. The grain is declared by a unique index inside one of sixty-two migrations. The four-hour de-duplication window that makes the published attendance figure differ from a raw count of check-ins lives in one method of AttendanceService.cs and is written down nowhere else. The rounding rule is a paragraph on page 9 of a PDF. The only person who ever wrote
anything about legacy_flag left a column comment in 2019 and has since left.

The definitions layer answered who decided what this means. Sources answers the question underneath it: where was it written down, and how do we know? It is the step before the catalogue, and it is the first place in v2 where Spiff itself is the thing being reviewed.

The object model

Four concepts, and they nest.

- A kind of source is a category of raw material — five of them — each with its own statement of what Spiff opens when it reads one. The kind is picked first in the wizard, because it changes everything that follows: what can be in scope, what a read will produce, and how much of it can be trusted.
- A source is one registered thing: a name, a vendor, a host or repository, an owner, a system of record where there is one, an explicit list of what is in scope and a second explicit list of what is deliberately excluded, an identity mode with a risk level, a state, a health strip, a yield, and a coverage figure. Nine are registered. Eight of them read something; the ninth has read nothing at all and is waiting on two named people.
- An understanding run is one read of one source, at a point in time. It records what triggered it, who or what started it, a manifest of what it opened, what it proposed, how long it took and what it cost. Nine runs are on record, and a run is never overwritten — the previous reading of a rule stays visible beside the one that replaced it.
- A finding is one thing a run concluded, written as a single sentence a non-engineer can judge, plus a paragraph of detail, a type, a confidence figure, a status, the named role that has to decide it, a statement of what accepting it would do, and an evidence array. Twenty-seven findings across the nine runs, carrying thirty-two pieces of evidence between them.

Evidence is the load-bearing part. Every item is a file, a migration, a page, a schema object, a column comment or a labelled scan probe, quoted verbatim with its exact reference and a short note
explaining what in the snippet supports the claim. Eighteen are source code, eight are documents, three are schema, one is a column comment, and two are probes that touched the source's own data —
those two are labelled as such on the finding, on the evidence, and in the log.

Five kinds, and what Spiff reads from each

Kind	What Spiff opens	Why it is registered separately
Database	Tables and views, primary and foreign keys, indexes, check constraints, stored procedures, column comments	The shape is authoritative and the comments are not. A schema says what a column is and never what it means.
Code repository	Migrations, ORM models, validators, service logic, tests, README and inline comments	Where the rules actually live. Six business rules and two agreed definitions trace to one repository alone, the four-hour de-duplication window among them — and it appears in no schema

and no document. |
Document library	Data dictionaries, policies, retention schedules, handbooks, and the tables inside them	The place a rule is supposed to be recorded. It is also the source that most often disagrees with the code.
Spreadsheets & files	Sheet and column headers, formulas, named ranges, types, and the notes people leave in the margins	Rarely tidy, often the only place a number exists. Our messiest source holds a July total that disagrees with the warehouse by 3.1%.
API	The OpenAPI or GraphQL spec, endpoint shapes, field types, enumerations, error contracts, rate limits	A contract describes shapes and never reasoning, which is why the API source has the lowest coverage in the estate and honestly says so.

Registering a code repository beside the database it serves is the whole point of the feature. The Assemble database yields 64 tables and 812 fields and explains none of them; the repository next door yields six tables and forty-one derived fields that never appear in the schema at all — and the definition of attendance.

Registering a source: five steps

Kind · Connect · Scope · Identity · Understanding. Two of the five are governance steps rather than plumbing.

Scope is a tick-list of what may be opened, plus a free-text block of things that are never opened whatever else changes — pastoral, care notes, safeguarding, counselling, payment, bank details, free-text notes on a person. Those exclusions are applied when the connection opens, not in the query, and they fail closed: a query that names an excluded schema errors rather than returning nothing, so a mistake is loud instead of silent.

Identity is the most consequential control on the form, and it is a two-card choice with the cost of each stated on the card. The last step ends with the promise the rest of the area exists to keep: a read starts now, everything it works out arrives as one plain sentence with the file it came from, and nothing enters the catalogue until a named person accepts it.

The finding lifecycle

Proposed → accepted, accepted with edits, or rejected. Four states, and the middle one earns its place.

- Accept writes the finding into the catalogue and records the acceptor's name against it for as long as it stands. The card says what acceptance will do before you do it — "accepting this writes the agreed definition of Attendance, which 9 saved answers already use; every one of them re-runs against the new wording."
- Edit changes the wording, never the evidence. The original proposal is kept beside the accepted version so the two can be compared, and the result is recorded as an edit rather than a plain acceptance — because "a human rewrote this" and "a human agreed with this" are different facts about a definition.
- Reject requires a reason in the person's own words. The reason travels with the finding for as long as the source is registered and is shown to the next run, so the same thing is not proposed again blind. A rejected finding can be reconsidered later; the rejection stays visible.

Bulk acceptance exists, because a steward facing twenty structural findings will not click twenty times. It is not a shortcut past the record: each finding is written to the activity log separately, with its own evidence reference, against the person who selected it.

A run cannot be signed off while anything is undecided. The sign-off button is present, visibly disabled, and says why — leaving a finding open is itself a decision, and an unrecorded one.

Confidence bands, and why weak has to look weak

Three bands: strong at 90 and above, corroborated by code and, where one exists, a written document; probable at 70–89, supported by one kind of evidence only; weak below 70, thin,

reported so a person can decide rather than proposed as fact. Of the twenty-seven findings, nineteen are strong, four probable and four weak.

A number is not enough. A weak finding renders differently: dashed left border instead of solid, a recessed ground, a warning callout that says "weak, and shown as weak — Spiff would rather hand you a thin finding honestly labelled than a confident one it cannot support." The findings list can be grouped by confidence as well as by kind, so a steward can work the strong ones quickly and
give the weak ones the attention they need.

The clearest case in the fixture is legacy_flag at 41%. Its entire evidence base is one column comment from 2019 with somebody's initials on it — and the evidence panel prints what is missing as plainly as what is there: no reader in 88 service classes, no test in 214, no dictionary entry. Showing the absence is what makes the number honest.

Every run also carries a "what it did not read, and why" block: the pastoral module that is never opened, the drafts folder that is skipped, the 206 column comments treated as weak evidence throughout, the three workbooks that would not open. A manifest is an evidence base, not a progress
bar.

Conflicts: Spiff shows both and refuses to guess

Conflict is a finding type of its own, and it renders as two claims side by side — one panel per source, each with its own evidence and its own "read the evidence" button — under a flat statement:
Spiff will not choose. Three of them carry the mockup:

- The code counts a Youth meeting as a Regular meeting; the data dictionary says Youth is counted separately. On July's figures the two readings differ by 1,842 meetings.
- The code treats a transferred member as active for thirty more days; the membership handbook says
membership ends the day the transfer is recorded. The grace period counts 1,206 members in two regions at once.
- The Statistics team's hand-kept July total is 3.1% higher than the warehouse figure — 651 members
in Western Cape alone, concentrated in localities with no door scanner.

None of the three has a technically correct answer. Each is a decision with a named owner, and until that person decides, every answer touching the number carries the gap and names both figures.
Confidence on a conflict is deliberately low — 58 to 68 — not because the evidence is thin but because two authoritative sources cannot both be right, and the score should reflect that.

What registering a source does and does not do

It grants nobody access to anything. That sentence sits at the foot of the Sources index in a callout, and it is the governance claim the whole area turns on. Registering a source lets Spiff read it to work out what the data means. Whether a particular person may see a particular row is decided every time they ask, by their own permissions and the rules in force. A source can be fully understood and still return nothing at all to someone who is not entitled to it.

Identity is stated per source, and one honest weak point is left in. Four modes: read as the person asking (the source itself proves the scoping, and its own logs name the individual); signed in per person (the vendor's audit trail names the individual, and a lapsed sign-in stops that person's answers and nobody else's); a read-only key scoped to one repository (code only, so there is no personal record to scope in the first place); and a shared service account. The warehouse runs on the last one, and the UI says exactly what that costs: per-viewer scoping cannot be proved here. Spiff still re-checks permissions and still logs the person who asked, but the source cannot corroborate it, and Snowflake's own logs show the service account. The source page

carries a persistent critical callout, every dataset traced to it carries the warning, and the exception is dated — replaced with per-user key pairs by December — rather than open-ended. The
Finance ERP registration is stuck in awaiting approval for the same reason: Sage X3 has no per-user read API, so switching it on is a decision two named people have to make, and nothing has
been read while it waits.

Every acceptance is written to the activity log. Accepted, edited or rejected, in bulk or one at a time, with the actor, the effective subject, the evidence reference and the statement itself. That closes the loop the Activity log was built for: the log already carried who read what, and it now also carries who decided what the data means.

Coverage measures understanding, not reach. The Sources index sorts worst-first on a coverage figure that answers "how much of what this source holds can Spiff explain?" — not "how much can it
read". A deliberate exclusion lowers it, and should. Directory sits at 84 because three schemas are denied at the connection and always will be; the statistics workbooks sit at 23 because nothing from them has ever been accepted. Neither number is a target.

How it threads into the screens that already existed

Sources is not a parallel universe; it is the upstream half of screens that were already there.

- Data catalogue — every dataset now has a provenance question attached, and the Sources index answers it in both directions. A side panel names the three of fifteen datasets that trace to no registered source at all: two loaded by hand with nothing keeping them current, and one that is excluded by policy and never will be loaded.
- Dataset profile — grain, joins and field meanings arrive here as accepted findings. The grain statement on the meetings dataset is a finding that cites a unique index; the note on checked_in about localities without a scanner is a finding a steward accepted with edits.
- Business rules — a rule's plain-English sentence is the finding's sentence. The "What it produced" tab on a source lists every rule traced to it and names the file each one came from, so a rule can be argued about on the evidence.
- Definitions / glossary — an accepted definition finding is what puts a term in the glossary with an owner, a version and a date. The finding card states the blast radius before acceptance: how many saved answers already use the term and will re-run against the new wording.
- Activity log — the destination for every accept, edit and rejection, using the existing vocabulary and the existing run-as column.
- Connectors (MCP) — deliberately separate. A connector is a tool Spiff can use; a source is material Spiff can read. Both screens state a ceiling, and the Sources ceiling is its own panel: never write to it, never copy it wholesale, never read anything on the exclusion list, never answer from a cache without re-checking who is asking.

Open questions

1. Will a steward read the evidence, or just the confidence score? The whole design assumes the snippet is the point and the percentage is a sorting aid. The opposite habit — accept anything above 90, ignore everything below — would produce exactly the unreviewed derived rules the feature exists to prevent.
2. Who is the named person, really? The mockup resolves three roles to three people and falls back to the dataset owner. A real build needs a routing rule that survives absence, handover and a steward with 40 pending findings.
3. What happens on the second run? A re-read of changed code will re-propose things already accepted, contradict them, or find that an accepted rule no longer exists in the code. We show that runs are never overwritten; we have not designed the diff between two readings of the same rule, and that is the screen a real build needs next.
4. How is a stale acceptance detected? An accepted finding is a snapshot of a file that keeps

moving. Certification decays when a dataset is edited; an accepted rule should decay when the code it cites changes, and nothing in the mockup does that yet.
5. Is coverage a number anyone should see? It rewards reading more and punishes excluding things, which is exactly backwards for pastoral material. The footnote says so; a real build may need two numbers, or none.
6. What does a real read actually cost? The runs carry plausible durations and rand figures because a finance director will ask. They are invented. The shape of the answer — reading is cheap, being wrong is not — is the part we are confident about.
7. Test fixtures, dead code and branches that never run. The mockup skips fixtures by configuration and says so. A real build has to distinguish a rule that runs from a rule that merely compiles, and nothing in a static read can tell the difference without production telemetry.

Decisions taken

1. Certification has four states plus one. Verified / Draft / Warning / Deprecated, because "risky right now" and "retired" are different messages — plus blocked for data excluded by policy. Pastoral care notes appear in the catalogue with no fields and a note saying so; deliberate absence is more trustworthy than silence.
2. Datasets carry the badge, answers inherit it. The dataset is what someone is asked to trust before asking a question.
3. Entitlements bind to groups, never to people. A person's row is read-only for access. It is the only model that survives 428 users, and it makes "why does she have this?" answerable.
4. Every grant has an end date, and every group says where it came from — synced, attribute rule, or manual with a review cadence.
5. Purpose is required on an access request. It is cheap to collect and it is the field an auditor reads.
6. Reads are logged. The Activity log is a product feature for end users, not an admin afterthought — you can see your own run history.
7. Preview-as-viewer is a product surface. It is the only way to make three separate promises legible at once, so it is a screen with a loud banner and its own audit events, not a hidden admin toggle.
8. "No data" and "not allowed to see the data" never look the same. Every empty state says which it is.
9. The interpretation is the headline. If Spiff has misread the question, that must be visible before the chart is.
10. Writes default to needing approval. Everywhere — connectors, automations, the composer.

Deferred

Drag-to-reorder and branch blocks in the workflow builder, still open from v1. Lineage beyond one hop. A dedicated feedback-review screen for downvoted answers — the research argues for one,
but it needs an owner-facing workflow we have not designed yet.